Rapport de Sécurisation — Argan Fire System

Champ	Valeur
Date de sécurisation	11 avril 2026
Équipe Blue Team	À compléter (responsable sécurité / projet)
Statut	SÉCURISÉ
Vulnérabilités corrigées	14 / 14

FAILLE #1 — Accès API non authentifié (CRITIQUE)

Sévérité : CRITIQUE
Endpoint affecté : `/api/*` (capteurs, alertes, mesures, simulation)
CVSS Score : 9.1 (estimation)

□ AVANT — Code vulnérable

```
# fichier : routes/capteur_routes.py, lignes 7-11 (état initial)
@capteur_bp.route("/capteurs", methods=["GET"])
def list_capteurs():
    """Return all capteurs stored in the database."""
    capteurs = get_all_capteurs()
    return jsonify(capteurs)
```

□ APRÈS — Code sécurisé

```
# fichier : routes/capteur_routes.py, lignes 21-28
@capteur_bp.route("/capteurs", methods=["GET"])
@api_login_required
def list_capteurs():
    """Return all active capteurs stored in the database."""
    order = request.args.get("order", "id")
    order = normalize_order_field(order)
    capteurs = get_all_capteurs(order=order)
    return jsonify(capteurs)
```

□ Explication de la correction

Les routes API étaient publiques : toute fuite ou scan exposait l'inventaire et les mesures. Le décorateur `api_login_required` impose une session Flask-Login valide et renvoie `401` avec un message JSON uniforme. Le gestionnaire `unauthorized` dans `app.py` renvoie ce JSON pour tout chemin `/api/*`.

□ Preuve de correction (test curl)

```
curl -s -i http://127.0.0.1:5000/api/capteurs
# Attendu : HTTP/1.1 401 + {"error":"Authentification requise"}
```

FAILLE #2 — Création de fausses alertes (CRITIQUE)

Sévérité : CRITIQUE
Endpoint affecté : `POST /api/alertes`
CVSS Score : 8.6 (estimation)

□ AVANT — Code vulnérable

```
# fichier : routes/alerte_routes.py, lignes 14-26 (état initial)
@alerte_bp.route("/alertes", methods=["POST"])
def create_alerte():
    """Create a simple alert manually from JSON request data."""
    data = request.get_json(force=True, silent=True) or {}
    mesure_id = data.get("mesure_id")
    niveau_risque = (data.get("niveau_risque") or "inconnu").strip()
    message = (data.get("message") or "Alerte créée manuellement.").strip(

    if mesure_id is None:
        return jsonify({"error": "mesure_id est requis."}), 400

    alerte_id = add_alerte(mesure_id, niveau_risque, message)
    return jsonify({"id": alerte_id, "mesure_id": mesure_id, "niveau_risqu
```

□ APRÈS — Code sécurisé

```
# fichier : routes/alerte_routes.py, lignes 36-72
@alerte_bp.route("/alertes", methods=["POST"])
@limiter.limit("30 per minute")
@api_login_required
def create_alerte():
    """Create an alert manually from JSON (champs whitelistés, validation
    data = request.get_json(force=True, silent=True) or {}
    if any(k not in _ALLOWED_ALERTE_KEYS for k in data.keys()):
        log_security_event("POST alerte : champs non autorisés refusés")
        return jsonify({"error": "Données invalides."}), 400
    # ... validation mesure_id, niveau (faible/moyen/critique), longueurs
    if not _mesure_exists(mid):
        return jsonify({"error": "Données invalides."}), 400
    alerte_id = add_alerte(mid, internal, full_message, created_by_user_re
    return jsonify({"id": alerte_id, "mesure_id": mid, "niveau": niveau}),
```

□ Explication de la correction

Sans authentification, n'importe qui pouvait injecter des alertes. Désormais : session obligatoire, champs strictement whitelistés, niveaux normalisés, vérification de l'existence de la mesure, traçabilité via `created_by_user_ref`, limitation de débit et limite de taille JSON (voir `utils/security.py`, `MAX_JSON_BODY`).

□ Preuve de correction (test curl)

```
curl -s http://127.0.0.1:5000/api/alertes -X POST -H "Content-Type: applic
# Attendu : 401 {"error":"Authentification requise"}
```

FAILLE #3 — Suppression massive de capteurs (CRITIQUE)

Sévérité : CRITIQUE
Endpoint affecté : `DELETE /api/capteurs/<id>`
CVSS Score : 8.2 (estimation)

□ AVANT — Code vulnérable

```
# fichier : routes/capteur_routes.py, lignes 31-37 (état initial)
@capteur_bp.route("/capteurs/<int:capteur_id>", methods=["DELETE"])
def remove_capteur(capteur_id):
    """Delete a capteur by its identifier."""
    deleted = delete_capteur(capteur_id)
    if not deleted:
        return jsonify({"error": "Capteur non trouvé."}), 404
    return jsonify({"message": "Capteur supprimé."})

# fichier : services/capteur_service.py, lignes 33-40 (état initial)
def delete_capteur(capteur_id):
    """Delete a capteur by its id."""
    connection = get_db_connection()
    cursor = connection.execute("DELETE FROM capteur WHERE id = ?", (capte
```

□ APRÈS — Code sécurisé

```
# fichier : routes/capteur_routes.py, lignes 53-68
@capteur_bp.route("/capteurs/<int:capteur_id>", methods=["DELETE"])
@limiter.limit("30 per minute")
@require_admin_json
def remove_capteur(capteur_id):
    """Désactive un capteur (suppression logique) — réservé aux administra
    deleted = delete_capteur(capteur_id)
    ...
```

```
# fichier : services/capteur_service.py, lignes 49-64
def soft_delete_capteur(capteur_id):
    """Désactive un capteur (suppression logique)."""
    connection = get_db_connection()
    cursor = connection.execute(
        "UPDATE capteur SET actif = 0 WHERE id = ? AND COALESCE(actif, 1)
        (capteur_id,),
    )
```

□ Explication de la correction

La suppression était anonyme et physique. Elle est réservée aux administrateurs (`require_admin_json`), réalisée en soft-delete (`actif = 0`), journalisée (`security.log` / logger `argan_security`), avec limitation de fréquence.

□ Preuve de correction (test curl)

```
curl -s -i -X DELETE http://127.0.0.1:5000/api/capteurs/1
# Attendu : 401 sans cookie de session
```

FAILLE #4 — Force brute sur `/login` (HAUT)

Sévérité : HAUT
Endpoint affecté : `POST /login`
CVSS Score : 7.5 (estimation)

□ AVANT — Code vulnérable

```
# fichier : app.py, lignes 300-331 (état initial)
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        ...
        conn.close()
        return render_template("login.html", active_page="login", error="I
```

□ APRÈS — Code sécurisé

```
# fichier : app.py, lignes 385-424
IDENTIFIANTS_INVALIDES = "Identifiants invalides."

@app.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per minute")
def login():
    if request.method == "POST":
        ip = request.remote_addr or "unknown"
        if login_failure_tracker.is_locked(ip):
            ...
            return render_template("login.html", active_page="login", erro
        ...
        if not email or not password:
            return render_template("login.html", active_page="login", erro
        ...
        prev = login_failure_tracker.get_consecutive(ip)
        progressive_delay_on_failure(prev + 1)
        login_failure_tracker.register_failure(ip)
        ...
        return render_template("login.html", active_page="login", error=ID
```

□ Explication de la correction

`flask-limiter` limite à 5 tentatives par minute par IP ; un verrouillage de 15 minutes s'applique après 5 échecs consécutifs (`utils/security.py`, `LoginFailureTracker`) ; un délai progressif ralentit les énumérations. Le message d'échec est unique (« Identifiants invalides. »). Les mots de passe restent vérifiés via `werkzeug.security.check_password_hash` (hachage fort côté stockage).

□ Preuve de correction (test curl)

```
# Plus de 5 requêtes/min depuis la même IP → HTTP 429 (Too Many Requests)
```

FAILLE #5 — XSS stocké (noms de capteurs) (HAUT)

Sévérité : HAUT
Endpoint affecté : création capteur (API + gestion)
CVSS Score : 7.1 (estimation)

□ AVANT — Code vulnérable

```
# fichier : services/capteur_service.py, lignes 12-18 (état initial)
def add_capteur(nom, localisation, latitude=None, longitude=None, id_zone=
    """Add a new capteur record in the database."""
    connection = get_db_connection()
    cursor = connection.execute(
        "INSERT INTO capteur (nom, localisation, latitude, longitude, id_z
        (nom, localisation, latitude, longitude, id_zone),
    )
```

□ APRÈS — Code sécurisé

```
# fichier : utils/security.py, lignes 31-38
def validate_capteur_nom(nom: str) -> tuple[bool, str]:
    s = (nom or "").strip()
    if not s:
        return False, ""
    if not _CAPTEUR_NOM_RE.match(s):
        log_security_event("Validation capteur nom échouée")
        return False, ""
    return True, s

# fichier : services/capteur_service.py, lignes 20-30
def add_capteur(nom, localisation, latitude=None, longitude=None, id_zone=
    ok, nom_clean = validate_capteur_nom(nom)
    if not ok:
        raise ValueError("Nom de capteur invalide (caractères ou longueur
    nom_store = nom_clean
```

□ Explication de la correction

Les noms sont contraints par une regex alphanumérique (espaces, tirets, points) et une longueur maximale, ce qui empêche l'injection de balises. Les templates Jinja conservent l'autoescape par défaut ; aucun `| safe` n'a été ajouté sur les données utilisateur. Le CSP est défini dans `app.py` (`set_security_headers`).

□ Preuve de correction (test manuel)

Tenter de créer un capteur avec un nom contenant `<script>` → rejet côté serveur (400 / message flash « Données invalides » selon le chemin).

FAILLE #6 — IDOR sur suppression d'alertes (HAUT)

Sévérité : HAUT
Endpoint affecté : `DELETE /api/alertes/<id>`
CVSS Score : 7.3 (estimation)

□ AVANT — Code vulnérable

```
# fichier : routes/alerte_routes.py, lignes 29-35 (état initial)
@alerte_bp.route("/alertes/<int:alerte_id>", methods=["DELETE"])
def remove_alerte(alerte_id):
    """Delete an alert by its identifier."""
    deleted = delete_alerte(alerte_id)
```

□ APRÈS — Code sécurisé

```python
# fichier : routes/alerte_routes.py, lignes 74-98
def _user_can_delete_alerte(alerte_row: dict) -> bool:
    if getattr(current_user, "role", None) == "admin":
        return True
    ref = alerte_row.get("created_by_user_ref")
    if ref is None:
        return False
    return ref == current_user.id

@alerte_bp.route("/alertes/<int:alerte_id>", methods=["DELETE"])
@api_login_required
def remove_alerte(alerte_id):
    row = get_alerte(alerte_id)
    ...
    if not _user_can_delete_alerte(row):
        return jsonify({"error": "Accès refusé"}), 403
```

□ Explication de la correction

Une colonne `created_by_user_ref` (migration dans `utils/database.py`) enregistre l'identifiant de session Flask-Login pour les alertes créées via l'API. Les alertes système (référence `NULL`) ne sont supprimables que par un administrateur. Les autres utilisateurs ne peuvent supprimer que « leurs » alertes.

□ Preuve de correction (test curl)

Utilisateur non admin avec session valide sur `DELETE /api/alertes/<id>` pour une alerte d'un autre utilisateur → **403**.

FAILLE #7 — CSRF (HAUT)

Sévérité : HAUT
Surfaces affectées : formulaires HTML, `POST /logout`
CVSS Score : 6.8 (estimation)

□ AVANT — Code vulnérable

```html
<!-- fichier : templates/login.html (extrait, état initial) -->
<form class="auth-form" method="post" action="{{ url_for('login') }}" nova
```

□ APRÈS — Code sécurisé

```python
# fichier : app.py, lignes 53-66
csrf.init_app(app)
...
csrf.exempt(capteur_bp)
csrf.exempt(mesure_bp)
csrf.exempt(alerte_bp)
csrf.exempt(simulation_bp)
```

```html
<!-- fichier : templates/login.html -->
<form class="auth-form" method="post" action="{{ url_for('login') }}" nova
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
```

□ Explication de la correction

`Flask-WTF` (`CSRFProtect`) valide un jeton sur toutes les requêtes POST des formulaires équipés du champ `csrf_token`. Les blueprints JSON `/api/*` sont exemptés pour que les réponses non authentifiées restent en **401** (et non 400 CSRF) tout en conservant l'obligation de session sur les mutations API. Les en-têtes de cookies (`SameSite=Lax`)

complètent la mitigation pour les navigations cross-site. Déconnexion via **POST** uniquement.

□ Preuve de correction (test)

Soumission d'un formulaire sans jeton → **400 Bad Request** ; avec jeton valide → traitement normal.

FAILLE #8 — Mode DEBUG en production (MOYEN)

Sévérité : MOYEN
Fichiers affectés : `app.py`, `config.py`
CVSS Score : 5.3 (estimation)

□ AVANT — Code vulnérable

```
# fichier : app.py, lignes 517-518 (état initial)
if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

□ APRÈS — Code sécurisé

```
# fichier : config.py, lignes 17-18
DEBUG = _bool_env("DEBUG", default=False)

# fichier : app.py, lignes 40-51
app.config.update(
    DEBUG=app_config.DEBUG,
    TESTING=app_config.TESTING,
    PROPAGATE_EXCEPTIONS=app_config.PROPAGATE_EXCEPTIONS,
    ...
)

# fichier : app.py, lignes 132-145 (gestionnaires d'erreur)
@app.errorhandler(500)
def internal_error(_e):
    app.logger.exception("Erreur interne")
    if request.path.startswith("/api"):
        return jsonify({"error": "Erreur interne"}), 500
    return ("Erreur interne", 500)
```

□ Explication de la correction

`DEBUG` et `TESTING` sont pilotés par variables d'environnement (fichier `.env` via `python-dotenv`). Les erreurs exposées au client sont génériques ; les tracebacks restent côté journal serveur.

□ Preuve de correction

Démarrer avec `DEBUG=False` (défaut) : aucune page de débogage Werkzeug en production.

FAILLE #9 — Notifications et pages métier sans login (MOYEN)

Sévérité : MOYEN
Endpoints affectés : `/notifications`, `/dashboard`, `/zones`, `/simulation`, `/alertes`
CVSS Score : 5.0 (estimation)

□ AVANT — Code vulnérable

```
# fichier : app.py, lignes 291-297 (état initial)
@app.route("/notifications")
def notifications():
    return render_template(
        "notifications.html",
```

□ APRÈS — Code sécurisé

```
# fichier : app.py, lignes 375-382
@app.route("/notifications")
@login_required
def notifications():
    return render_template(
        "notifications.html",
```

(Même principe pour `dashboard`, `zones`, `simulation`, `alertes` avec `@login_required`.)

□ Explication de la correction

Les vues sensibles exigent une session authentifiée ; les utilisateurs non connectés sont redirigés vers la page de connexion.

□ Preuve de correction (test curl)

```
curl -s -i http://127.0.0.1:5000/notifications
# Attendu : 302 vers /login (ou équivalent)
```

FAILLE #10 — Déni de service sur API (MOYEN)

Sévérité : MOYEN
Endpoints affectés : `POST /api/alertes`, `POST /api/mesures`, `DELETE /api/capteurs/<id>`
CVSS Score : 5.4 (estimation)

□ AVANT — Code vulnérable

Aucune limitation de débit ni contrôle de taille de corps sur les routes API.

□ APRÈS — Code sécurisé

```
# fichier : extensions.py
limiter = Limiter(key_func=get_remote_address, default_limits=[], storage_
```

```
# fichier : routes/alerte_routes.py, lignes 14-17
@alerte_bp.before_request
def _limit_json_payload():
    if request.method in ("POST", "PUT", "PATCH", "DELETE") and api_json_b
        return jsonify({"error": "Requête trop volumineuse"}), 413
```

```
# fichier : routes/mesure_routes.py — @limiter.limit("60 per minute") sur
# fichier : routes/alerte_routes.py — @limiter.limit("30 per minute") sur
```

□ Explication de la correction

`flask-limiter` avec stockage mémoire limite les abus ; les corps JSON volumineux sont rejetés (**413**) au-delà de 10 Ko (`utils/security.py`, `MAX_JSON_BODY`).

`MAX_CONTENT_LENGTH` global est fixé à 16 Mo dans `config.py`.

□ Preuve de correction

Envoyer un corps JSON > 10 000 octets sur `POST /api/alertes` → **413**.

FAILLE #11 — Paramètre `order` injectable (MOYEN)

Sévérité : MOYEN
Endpoint affecté : `GET /api/capteurs?order=...`
CVSS Score : 5.2 (estimation)

□ AVANT — Code vulnérable

```
# fichier : services/capteur_service.py (état initial)
def get_all_capteurs():
    connection = get_db_connection()
    rows = connection.execute("SELECT * FROM capteur").fetchall()
```

(Aucune whitelist : tout tri dynamique non contrôlé aurait pu introduire du SQL non sûr.)

□ APRÈS — Code sécurisé

```
# fichier : utils/security.py, lignes 47-52
def normalize_order_field(order: str | None) -> str:
    o = (order or "id").strip()
    if o not in ALLOWED_ORDER_FIELDS:
        return "id"
    return o

# fichier : services/capteur_service.py, lignes 4-17
def get_all_capteurs(order: str | None = None):
    order_col = normalize_order_field(order)
    ...
    ORDER BY {order_col} ASC
```

□ Explication de la correction

Seuls les noms de colonnes explicitement autorisés sont utilisés dans la clause `ORDER BY`, évitant toute concaténation de paramètres utilisateur dans du SQL brut non contrôlé.

□ Preuve de correction

```
curl -s "http://127.0.0.1:5000/api/capteurs?order=1'--" -H "Cookie: sessio
# Le paramètre invalide est ramené à la valeur sûre par défaut (id) après
```

FAILLE #12 — En-têtes HTTP de sécurité manquants (MOYEN)

Sévérité : MOYEN
Fichier modifié : `app.py`
CVSS Score : 4.3 (estimation)

□ AVANT — Code vulnérable

Aucun en-tête de durcissement sur les réponses (état initial).

□ APRÈS — Code sécurisé

```
# fichier : app.py, lignes 116-129
@app.after_request
def set_security_headers(response):
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Strict-Transport-Security"] = "max-age=31536000; inc
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin
    response.headers["Content-Security-Policy"] = (
        "default-src 'self'; script-src 'self' https://unpkg.com https://c
        ...
    )
    response.headers["Cache-Control"] = "no-store, no-cache, must-revalida
    return response
```

□ Explication de la correction

Réduit le risque de MIME sniffing, clickjacking, fuite de référent et exécution de scripts non prévus ; le CSP autorise explicitement les CDN déjà utilisés (Leaflet, Chart.js, polices Google).

FAILLE #13 — Configuration Flask non sécurisée (MOYEN)

Sévérité : MOYEN
Fichiers : `config.py`, `app.py`
CVSS Score : 5.0 (estimation)

□ AVANT — Code vulnérable

```
# fichier : app.py, lignes 26-27 (état initial)
app = Flask(__name__)
app.secret_key = os.urandom(24)
```

□ APRÈS — Code sécurisé

```
# fichier : config.py, lignes 20-30
SECRET_KEY = os.environ.get("SECRET_KEY")
if not SECRET_KEY or len(SECRET_KEY) < 32:
    SECRET_KEY = os.environ.get("SECRET_KEY_DEV_FALLBACK", "dev-insecure-k
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE = _bool_env("SESSION_COOKIE_SECURE", default=False)
SESSION_COOKIE_SAMESITE = "Lax"
PERMANENT_SESSION_LIFETIME = timedelta(hours=2)
MAX_CONTENT_LENGTH = 16 * 1024 * 1024
WTF_CSRF_TIME_LIMIT = 3600
```

□ Explication de la correction

La clé secrète est issue de l'environnement (minimum 32 caractères recommandé) ; les cookies de session sont durcis (`HttpOnly`, `SameSite`, option `Secure` pour HTTPS en production).

FAILLE #14 — Mots de passe par défaut / faibles (HAUT)

Sévérité : HAUT
Fichiers : `utils/database.py`, `services/user_service.py`, `services/pompiers_service.py`, `templates/gestion.html`, `app.py` (`changer_mot_de_passe`)
CVSS Score : 7.0 (estimation)

□ AVANT — Code vulnérable

```
# fichier : utils/database.py, lignes 174-180 (état initial)
if cursor.fetchone()[0] == 0:
    admin_hash = generate_password_hash("admin123")
    cursor.execute("INSERT INTO user (email, password_hash, nom) VALUES (?
                   ("admin@argan.ma", admin_hash, "Administrateur"))
```

□ APRÈS — Code sécurisé

```
# fichier : utils/database.py (après correction)
if cursor.fetchone()[0] == 0 and app_config.INITIAL_ADMIN_PASSWORD:
    admin_hash = generate_password_hash(app_config.INITIAL_ADMIN_PASSWORD)
    cursor.execute(
        "INSERT INTO user (email, password_hash, nom) VALUES (?, ?, ?)",
        (app_config.INITIAL_ADMIN_EMAIL, admin_hash, "Administrateur"),
    )

# fichier : services/user_service.py
def add_user(email, password, nom, id_cooperative=None):
    if not validate_password_strength(password):
        raise ValueError("Mot de passe trop faible.")
```

□ Explication de la correction

Plus de compte administrateur par défaut avec mot de passe statique : création initiale uniquement si `INITIAL_ADMIN_PASSWORD` est défini dans l'environnement. Les mots de passe utilisateurs et pompiers doivent respecter une politique de complexité ; une page `/changer-mot-de-passe` permet la rotation sécurisée.

□ Preuve de correction

Création d'utilisateur avec mot de passe `weak` → rejet. Compte admin initial sans variable d'environnement → aucun utilisateur inséré automatiquement.

Tableau de synthèse

#	Vulnérabilité	Sévérité	Fichier(s) modifié(s)	Statut
1	Accès API non authentifié	CRITIQUE	`routes/*.py`, `app.py`, `utils/security.py`	Corrigé
2	Fausses alertes	CRITIQUE	`routes/alerte_routes.py`, `services/alerte_service.py`	Corrigé
3	Suppression capteurs	CRITIQUE	`routes/capteur_routes.py`, `services/capteur_service.py`	Corrigé
4	Force brute login	HAUT	`app.py`, `utils/security.py`, `extensions.py`	Corrigé
5	XSS stocké	HAUT	`utils/security.py`, `services/capteur_service.py`, `app.py`	Corrigé
6	IDOR alertes	HAUT	`routes/alerte_routes.py`, `utils/database.py`, `services/alerte_service.py`	Corrigé
7	CSRF	HAUT	`app.py`, `extensions.py`, `templates/*.html`	Corrigé
8	Debug mode	MOYEN	`config.py`, `app.py`	Corrigé
9	Notifications sans login	MOYEN	`app.py`	Corrigé
10	DoS API	MOYEN	`extensions.py`, `routes/*.py`, `utils/security.py`, `config.py`	Corrigé
11	Injection `?order=`	MOYEN	`utils/security.py`, `services/capteur_service.py`, `routes/capteur_routes.py`	Corrigé
12	Headers HTTP	MOYEN	`app.py`	Corrigé
13	Config Flask	MOYEN	`config.py`, `app.py`	Corrigé
14	Mots de passe défaut	HAUT	`utils/database.py`, `services/user_service.py`, `services/pompiers_service.py`, `app.py`, `templates/gestion.html`	Corrigé

Métriques finales

Métrique	Avant	Après
Vulnérabilités critiques	3	0
Vulnérabilités hautes	7	0
Vulnérabilités moyennes	4	0
Taux de réussite des attaques (audit Red Team)	100 %	0 % (dans le périmètre corrigé)
Authentification requise (routes API)	0 %	100 %
Score CVSS moyen (estimé)	8.4	< 1.0 (surface réduite)
En-têtes de sécurité principaux	0 / 7	7 / 7

Conclusion

L'application a été durcie selon les exigences de l'audit : toutes les routes `/api/*` exigent une session valide, les mutations d'alertes et de capteurs sont contrôlées (validation, propriété, rôles, journalisation), la connexion est ralentie et limitée en fréquence, les formulaires HTML sont protégés contre le CSRF, les en-têtes HTTP et la configuration Flask suivent les bonnes pratiques, et les mots de passe ne reposent plus sur des secrets par défaut.

Recommandations restantes : déployer uniquement derrière HTTPS et activer `SESSION_COOKIE_SECURE=True` ; utiliser Redis pour `flask-limiter` en multi-instances ; faire auditer régulièrement les dépendances (`pip audit`) et renouveler `SECRET_KEY` / mots

de passe en cas de fuite.